

Google Cloud Professional Cloud Architect (PCA) Study Guide

This guide is organized by the official exam domains (based on the current exam guide, effective for exams on/after the update referenced in Google's v6.1 materials). Percentages are approximate and can shift slightly. Familiarity with the **Google Cloud Well-Architected Framework** is explicitly required and woven throughout.

Exam format reminder: 50–60 multiple-choice/multiple-select questions, 2 hours, case studies available on a split screen. Case-study questions form a significant portion of the exam.

Google Cloud Well-Architected Framework

The framework provides principles and recommendations for designing, building, and operating workloads that are reliable, secure, efficient, cost-optimized, and sustainable. Its six pillars guide nearly every architectural decision on the exam:

- **Operational excellence** — Efficiently deploy, operate, monitor, and manage workloads. Focus on automation, IaC, observability, change management, and continuous improvement.
- **Security, privacy, and compliance** — Maximize security and privacy; align with regulations. Emphasize security by design, least privilege, zero trust, encryption, and secure AI.
- **Reliability** — Design and operate resilient, highly available systems. Use redundancy, horizontal scaling, realistic targets (SLOs/SLIs), observability, graceful degradation, and recovery testing.
- **Cost optimization** — Maximize business value. Align spending with value, foster cost awareness, right-size resources, use committed use discounts (CUDs)/spot VMs, and continuously optimize.
- **Performance optimization** — Design and tune for optimal performance. Plan resource allocation, enable elasticity, use modular design, and measure continuously.
- **Sustainability** — Build environmentally sustainable workloads (carbon-aware choices, efficient resource use).

Cross-pillar perspectives include AI/ML and industry-specific guidance (e.g., financial services). Apply the framework to every design trade-off: cost vs. performance, availability vs. complexity, security vs. usability, etc.

Links: Every domain below maps back to one or more pillars. Exam answers that ignore Well-Architected principles (e.g., over-provisioning without justification, weak IAM, missing

observability) are usually incorrect.

1. Designing and planning a cloud solution architecture (~25%)

Key concepts and services

- Translate business requirements (use cases, product strategy, KPIs/ROI, cost targets, compliance) and non-functional requirements (availability, latency, scalability, durability) into architecture.
- Compute choices: Compute Engine (custom machine types, spot VMs, sole-tenant), GKE (Autopilot/Standard), Cloud Run / Cloud Run functions, App Engine, specialized (AI Hypercomputer, GPUs/TPUs).
- Storage: Cloud Storage (classes, lifecycle), Cloud SQL, Cloud Spanner, Bigtable, Firestore, BigQuery, Filestore, Persistent Disk.
- Networking: VPC, Shared VPC, VPC peering, Private Service Connect, Cloud Load Balancing (global/regional, external/internal), Cloud CDN, Cloud Armor, Cloud Interconnect / Cloud VPN, Cloud NAT.
- Data movement & integration: Pub/Sub, Dataflow, Dataproc, Transfer Service, Migration Center.
- AI/ML: Vertex AI (Pipelines, Model Garden, Gemini models, Agent Builder), AI Hypercomputer.
- Migration: assess with Migration Center, choose strategy (lift-and-shift, rehost, replatform, refactor, retire, retain), license implications.
- Observability and business continuity from day one.

Important design trade-offs

- Cost vs. performance/availability (spot vs. standard, regional vs. multi-regional, Autopilot vs. Standard GKE).
- Managed vs. self-managed (Cloud SQL vs. self-managed MySQL on GCE; Cloud Run vs. GKE).
- Consistency vs. latency/scalability (Spanner vs. Cloud SQL vs. Bigtable).
- Hybrid/multi-cloud complexity vs. pure cloud-native simplicity.
- Build vs. buy vs. modify vs. deprecate workloads.

Best practices

- Start with business outcomes and map to Well-Architected pillars.

- Prefer managed, serverless, or Autopilot where possible.
- Design for horizontal scaling and multi-zone/region from the start.
- Use Shared VPC + hierarchical firewall policies + organization policies for governance.
- Document decisions with architecture diagrams and success metrics.
- Plan for future evolution (cloud-first, new AI capabilities).

Common exam scenarios and pitfalls

- Scenario: “Business needs 99.99% availability and global users.” Prefer multi-regional services + global load balancing; avoid single-region unless latency or data residency forces it.
- Pitfall: Choosing the “most powerful” option without cost or operational justification.
- Pitfall: Ignoring data gravity, egress costs, or license portability during migration.
- Pitfall: Overlooking integration patterns with external/on-prem systems.

Links to other domains: Feeds directly into provisioning (Domain 2), security (Domain 3), and operations (Domain 6). Migration plans must address compliance and reliability.

2. Managing and provisioning a solution infrastructure (~18%)

Key concepts and services

- Network topologies: hybrid (Interconnect, HA VPN), multi-cloud, Shared VPC, Private Google Access, Private Service Connect, firewall rules / hierarchical policies, load balancers.
- Storage configuration: allocation, lifecycle policies, retention, access controls, transfer/latency optimization, backup/recovery.
- Compute provisioning: machine types, spot vs. standard, GKE node pools / Autopilot, Cloud Run concurrency/CPU allocation, patch management, container orchestration.
- Vertex AI end-to-end: Pipelines for ML lifecycle, data integration, AI Hypercomputer (GPUs/TPUs, large-scale training), Model Garden integration, Gemini Enterprise / NotebookLM / AI Agents.
- Pre-built AI APIs (Vision, Video, Speech, etc.) vs. custom models.
- Infrastructure orchestration: Terraform / Infrastructure-as-Code, Deployment Manager (legacy), Config Connector, Anthos/Config Management where relevant.

Important design trade-offs

- Connectivity cost/performance (Dedicated Interconnect vs. Partner Interconnect vs. VPN).
- Storage class and location vs. access patterns and cost.
- Spot VMs / preemptible for batch vs. reliability needs.
- Managed Prometheus / Cloud Monitoring vs. self-managed observability stacks.

Best practices

- Use Shared VPC for centralized networking and security.
- Apply least-privilege service accounts and Workload Identity.
- Automate everything with IaC; avoid manual console changes in production.
- Configure data lifecycle and retention early.
- For AI: use Vertex AI Pipelines + Model Armor + Sensitive Data Protection.

Common exam scenarios and pitfalls

- Hybrid connectivity for low-latency or high-volume data (media ingestion, manufacturing telemetry). Prefer Interconnect + Private Service Connect.
- Pitfall: Public IPs or overly permissive firewall rules.
- Pitfall: Forgetting regional vs. multi-regional implications for storage and compute.
- Pitfall: Not planning for data growth or backup/DR in storage design.

Links: Builds on Domain 1 designs; security controls from Domain 3 must be applied during provisioning; observability from Domain 6 is configured here.

3. Designing for security and compliance (~19%)

Key concepts and services

- IAM: roles (primitive, predefined, custom), service accounts, Workload Identity Federation, Identity-Aware Proxy (IAP), context-aware access, organization policies.
- Resource hierarchy: Organization → Folders → Projects; hierarchical firewall policies.
- Data security: encryption (Google-managed, CMEK via Cloud KMS, CSEK), Secret Manager, VPC Service Controls, DLP / Sensitive Data Protection.
- Security controls: Cloud Audit Logs, Cloud Armor, Binary Authorization, Model Armor (for AI), secure software supply chain.
- Secure remote access: IAP, Chrome Enterprise Premium, service-account impersonation.

- Compliance: data residency/sovereignty, HIPAA, PCI-DSS, GDPR/CCPA-style privacy, SOC 2, children's privacy, commercial sensitive data. Audits and logging.

Important design trade-offs

- Least privilege vs. operational agility.
- CMEK vs. Google-managed keys (control vs. simplicity/cost).
- Perimeter security (VPC-SC) vs. usability of Google APIs.
- Centralized vs. decentralized IAM administration.

Best practices

- Enforce organization policies and hierarchical controls from day one.
- Prefer Workload Identity over long-lived keys.
- Encrypt sensitive data at rest and in transit; use DLP for classification.
- Secure the AI supply chain and runtime (Model Armor).
- Log everything relevant and retain per compliance needs.
- Separation of duties via IAM and resource hierarchy.

Common exam scenarios and pitfalls

- Healthcare (EHR) or retail (payments) → strong focus on encryption, VPC-SC, audit logs, and least privilege.
- Pitfall: Granting broad roles (Owner/Editor) or using user accounts for workloads.
- Pitfall: Exposing services publicly when IAP or Private Service Connect is available.
- Pitfall: Ignoring data residency or industry-specific regulations.

Links: Security is a cross-cutting concern for every domain. Compliance requirements drive choices in Domains 1, 2, and 4.

4. Analyzing and optimizing technical and business processes (~15%)

Key concepts and services

- Technical processes: SDLC, CI/CD (Cloud Build, Cloud Deploy, Artifact Registry, Binary Authorization), testing (unit/integration/load), troubleshooting/root-cause analysis, service catalog, disaster recovery.

- Business processes: stakeholder management, change management, team skills assessment, decision-making, customer success, CapEx/OpEx optimization, business continuity.
- Reliability practices: chaos engineering, penetration testing, load testing.
- Tools: Gemini Cloud Assist, Migration Center, cost management tools, operations suite.

Important design trade-offs

- Speed of delivery vs. risk (aggressive CI/CD vs. heavy governance).
- Build vs. buy for tooling.
- Centralized vs. federated process ownership.

Best practices

- Automate CI/CD and infrastructure with IaC and policy-as-code.
- Define clear SLOs/SLIs and error budgets.
- Treat cost as a first-class concern (FinOps practices).
- Continuously evaluate and improve processes using Well-Architected reviews.
- Involve stakeholders early; manage change and skills gaps.

Common exam scenarios and pitfalls

- “How do we accelerate releases while maintaining reliability?” → Cloud Build + Cloud Deploy + canary/blue-green + monitoring.
- Pitfall: Ignoring people/process side (skills, change management) when recommending technology.
- Pitfall: Recommending pure lift-and-shift when refactoring yields better long-term value.

Links: Feeds into implementation (Domain 5) and operations excellence (Domain 6). Optimization loops back to Domain 1 designs.

5. Managing implementations of cloud architecture (~11%)

Key concepts and services

- Advising teams: stakeholder management, change management, skills readiness, decision processes, customer success, cost/resource optimization.

- Programmatic interaction: Cloud Shell / Cloud Code, gcloud / gsutil / bq, Google Cloud SDKs and client libraries, Cloud Emulators, Terraform / IaC, Apigee for API management, Gemini Cloud Assist.
- Testing frameworks, data/system migration tooling.

Important design trade-offs

- Speed of implementation vs. long-term maintainability.
- Tooling standardization vs. team preference.

Best practices

- Prefer IaC and GitOps.
- Use Cloud Shell / Cloud Code for consistent developer experience.
- Provide clear runbooks and knowledge transfer.
- Continuously measure success against original KPIs/ROI.

Common exam scenarios and pitfalls

- Scenario: “Development team is struggling with local testing.” Recommend Cloud Emulators or Cloud Code.
- Pitfall: Recommending manual console operations for production changes.
- Pitfall: Ignoring organizational change and training needs.

Links: Bridges design (Domain 1) and ongoing operations (Domain 6).

6. Ensuring solution and operations excellence (~12%)

Key concepts and services

- Operational excellence pillar of the Well-Architected Framework.
- Observability: Cloud Monitoring, Cloud Logging, Cloud Trace, Cloud Profiler, Error Reporting, Managed Service for Prometheus, alerting strategies.
- Deployment and release management (Cloud Deploy, progressive delivery).
- Supporting deployed solutions, quality control, reliability practices (chaos engineering, load/penetration testing).

Important design trade-offs

- Depth of observability vs. cost and noise.
- Automated remediation vs. human-in-the-loop.

Best practices

- Define SLIs/SLOs and error budgets early.
- Instrument everything; use structured logging and distributed tracing.
- Automate responses where safe; maintain clear escalation paths.
- Conduct regular postmortems and Well-Architected reviews.
- Test recovery procedures (not just backups).

Common exam scenarios and pitfalls

- “How do we detect and recover from failures quickly?” → comprehensive monitoring + alerting + automated failover + chaos testing.
- Pitfall: Monitoring only infrastructure metrics while ignoring application SLIs.
- Pitfall: No defined recovery objectives or untested DR plans.

Links: Closes the loop with Domain 1 (design for operability) and Domain 4 (process optimization).

Official Case Studies – Summary and Recommended Architectures

The four current official case studies (available during the exam) emphasize modernization, hybrid connectivity, generative AI, and industry-specific constraints. Always map answers to business requirements, technical constraints, and Well-Architected principles.

1. Altostrat Media

- **Overview:** Media company with large audio/video library (podcasts, interviews, news, documentaries). Already using GKE, Cloud Storage, BigQuery, and Cloud Run functions. Some legacy on-prem systems for ingestion/archiving. Goal: modernize content management and user engagement with generative AI (personalized recommendations, natural-language interaction, 24/7 support, metadata extraction, content moderation).
- **Key requirements:** High availability for streaming, cost-efficient storage growth, hybrid ingestion, AI integration without disrupting existing services, appropriate/explainable AI outputs.

- **Recommended architecture highlights:**

- Event-driven pipeline: Cloud Storage → Cloud Run functions / Dataflow for transcoding, metadata, moderation (Video Intelligence + Vision + Gemini).
- Analytics & personalization in BigQuery + Vertex AI / Gemini.
- Hybrid: Dedicated Interconnect or HA VPN + Private Service Connect.
- Observability: Cloud Monitoring + Managed Service for Prometheus.
- Security: VPC-SC, CMEK where needed, Model Armor.
- **Exam focus:** Event-driven media pipelines, GenAI integration, hybrid connectivity, storage cost optimization, observability modernization.

2. Cymbal Retail

- **Overview:** Growing online retailer with large multi-category catalog. Hybrid environment (on-prem + cloud) with siloed databases (MySQL, SQL Server, Redis, MongoDB), legacy SFTP/batch integrations, IVR/phone support. Goals: catalog/content enrichment with GenAI, conversational commerce (AI virtual agents), personalized shopping, omnichannel inventory, PCI-DSS compliance.
- **Key requirements:** Unified customer/product view, low-latency inventory, secure payment handling, scalable e-commerce, reduced operational cost via automation.
- **Recommended architecture highlights:**
 - Modern data layer: Cloud Spanner (or AlloyDB) for global inventory consistency; BigQuery for analytics; Memorystore for caching.
 - Frontend: GKE or Cloud Run microservices.
 - AI: Vertex AI / Gemini for catalog enrichment, Discovery AI / Vertex AI Search + Dialogflow CX or Agents for conversational commerce; Model Armor.
 - Connectivity: Interconnect for stores + Private Service Connect.
 - Security/compliance: VPC Service Controls, DLP, tokenization, CMEK, strict IAM.
- **Exam focus:** Data modernization & consistency, GenAI for retail, hybrid inventory, PCI-DSS patterns, cost-aware scaling for seasonal traffic.

3. EHR Healthcare

- **Overview:** Leading SaaS provider of electronic health record software to multi-national medical offices, hospitals, and insurers. Currently in multiple colocation facilities (one lease expiring). Many customer-facing apps already containerized on Kubernetes. Goals: scale for rapid growth, improve DR, enable fast continuous deployment, reduce latency, maintain strict regulatory compliance (health-record privacy), provide consistent multi-environment management, centralized observability.
- **Key requirements:** 99.9%+ availability, multi-environment consistency, secure hybrid connectivity, fast onboarding of new providers, predictive insights, lower admin cost.
- **Recommended architecture highlights:**
 - Compute: GKE (preferably multi-cluster / multi-region or Anthos-style consistency) or Cloud Run for appropriate services.
 - Data: Migrate relational to Cloud SQL / Spanner / AlloyDB; use BigQuery for analytics/trends; Memorystore where caching is needed.
 - Networking: Dedicated Interconnect or HA VPN + Shared VPC + Private Service Connect.
 - CI/CD: Cloud Build + Cloud Deploy + Binary Authorization + Artifact Registry.
 - Observability: Cloud Operations suite + consistent logging/monitoring/alerting across environments.
 - Security/compliance: Strong IAM hierarchy, CMEK, VPC-SC, audit logs, data residency controls, HIPAA-aligned practices.
 - DR: Multi-region with tested failover, RPO/RTO aligned to business.
- **Exam focus:** Container platform consistency, hybrid connectivity, healthcare compliance, CI/CD + reliability, multi-region DR, observability.

4. KnightMotives Automotive

- **Overview:** Car manufacturer (BEVs, hybrids, ICE, autonomous vehicles). Heavy legacy (mainframe supply chain, outdated ERP, fragmented codebases). Manufacturing plants, rural connectivity challenges. Goals: modernize consumer experience with AI (in-vehicle, shopping, buying, service), consistent experience across models, data monetization, better tools for mechanics/sales, hybrid-cloud strategy, network upgrades.
- **Key requirements:** High-volume telemetry ingestion, edge/near-edge processing, hybrid modernization of legacy systems, AI features, secure data sharing, improved plant-to-HQ connectivity.

- **Recommended architecture highlights:**

- Telemetry: IoT Core or Pub/Sub → Bigtable (hot path) + BigQuery (analytics) or Dataflow.
- Edge: Google Distributed Cloud or edge devices + Vertex AI for on-vehicle/near-vehicle inference.
- Hybrid: Interconnect / SD-WAN-style connectivity + progressive modernization (strangler pattern for mainframe/ERP).
- AI/ML: Vertex AI + Model Garden / Gemini for in-vehicle and customer experiences; MLOps with Pipelines.
- Data platform: Central lakehouse pattern with BigQuery + appropriate operational stores.
- Security: Strong identity (including vehicle/device identity), encryption, least privilege across hybrid estate.
- **Exam focus:** High-throughput IoT/telemetry architecture, hybrid/legacy modernization, edge AI, automotive data patterns, network modernization.

General case-study approach on the exam

1. Identify the primary business goals and constraints (compliance, latency, cost, hybrid, AI).
 2. Map to the most appropriate Google Cloud services and Well-Architected principles.
 3. Prefer managed services, least privilege, and automation.
 4. Justify trade-offs explicitly (why not the simpler/cheaper option?).
 5. Ensure observability, security, and reliability are never afterthoughts.
-

Study tips

- Master the official exam guide and the four case-study PDFs.
- Practice architecture diagrams and decision trees for compute/storage/network/AI choices.
- Hands-on: build small solutions with GKE, Cloud Run, Spanner/Bigtable, Vertex AI, Shared VPC, Terraform, and the operations suite.
- Review cost calculators, SLOs, and common anti-patterns.
- Take practice exams that include the current case studies and focus on “why” explanations.

This structure covers every major topic in the official domains while emphasizing the

interconnections and the Well-Architected Framework that Google expects architects to apply. Good luck—focus on business outcomes, justified trade-offs, and secure, operable designs